

Application de Suivi de Traitements Médicaux

Rapport de Mini-Projet

Réalisé par :

EL-GHOMRY Youssef

DAOUDI Mouad

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM

Année : 2025/2026

Table des matières

1. Introduction

2. Présentation du projet

2.1 Description de l'application

2.2 Besoins fonctionnels

2.3 Besoins non fonctionnels

3. Conception

3.1 Architecture MVC

3.2 Diagramme de classes UML

3.3 Diagramme de cas d'utilisation

3.4 Diagramme de séquence

4. Environnement de travail

4.1 Outils et technologies utilisés

4.2 Structure du projet

5. Répartition des tâches

5.1 Tâches de l'étudiant 1 – EL-GHOMRY Youssef

5.2 Tâches de l'étudiant 2 – DAOUDI Mouad

6. Explication du code

6.1 Connexion à la base de données – Database.java

6.2 Les classes modèles

6.3 Les classes DAO

6.4 La vue principale – MainView.java

6.5 La fonctionnalité d'export

6.6 Les notifications Toast

7. Interfaces de l'application

7.1 Fenêtre principale

7.2 Gestion des patients

7.3 Gestion des traitements

7.4 Tableau de bord – Statistiques

7.5 Menus et dialogues

7.6 Export de données

8. Conclusion

9. Références

1. Introduction

La gestion des traitements médicaux est un enjeu crucial dans les établissements de santé. Ce projet vise à fournir une application desktop intuitive, développée en JavaFX, permettant de suivre les patients, leurs traitements et leur état de surveillance. L'application s'appuie sur une base de données MySQL pour assurer la persistance des données et offre des fonctionnalités CRUD, des statistiques et des exports CSV.

Ce rapport détaille les étapes de conception, de développement et d'implémentation de l'application, en suivant une architecture MVC et le pattern DAO.

2. Présentation du projet

2.1 Description de l'application

L'application « Suivi de Traitements Médicaux » permet aux professionnels de santé de :

- Gérer les dossiers patients (nom, prénom, date de naissance, sexe, téléphone, email, surveillance, observations).
- Associer à chaque patient un ou plusieurs traitements (nom, type, posologie, prises par jour, dates, durée, statut actif, effets secondaires, couleur).
- Visualiser la progression des traitements via des barres de progression.
- Obtenir des statistiques instantanées (nombre total de patients, traitements actifs, patients sous surveillance).
- Exporter les données vers des fichiers CSV.

L'interface est organisée en onglets et bénéficie d'un thème moderne, lisible et responsive.

2.2 Besoins fonctionnels

Fonctionnalité	Description
CRUD Patients	Ajouter, modifier, supprimer, rechercher et lister les patients.
CRUD Traitements	Ajouter, modifier, supprimer des traitements pour un patient donné.
Filtrage	Filtrer les traitements par type ; rechercher un patient par nom, prénom, téléphone ou email.
Statistiques	Afficher le nombre total de patients, de traitements actifs et de patients sous surveillance.
Export CSV	Exporter la liste des patients ou des traitements au format CSV.
Notifications	Affichage de toasts pour confirmer les actions (ajout, modification, suppression, erreur).
Thème	Interface cohérente avec un thème personnalisé (CSS).

2.3 Besoins non fonctionnels

- **Performance** : réactivité des interactions (chargement des données, recherche).
- **Fiabilité** : gestion des erreurs SQL et validations des champs obligatoires.
- **Maintenabilité** : code structuré selon le pattern DAO et MVC.
- **Sécurité** : utilisation de requêtes préparées pour éviter les injections SQL.

- **Ergonomie** : navigation par onglets, raccourcis clavier (Ctrl+N, Delete, Ctrl+Q, Ctrl+F, Échap), messages d'aide.

3. Conception

3.1 Architecture MVC

L'application suit le patron **Modèle-Vue-Contrôleur** :

- **Modèle** : les classes Patient et Traitement (package model) représentent les données métier.
- **Vue** : la classe MainView construit l'interface utilisateur (fenêtre, onglets, tableaux, dialogues) ; le thème est défini dans `medical-theme.css`.
- **Contrôleur** : la logique métier est répartie entre les DAO (PatientDAO, TraitementDAO) et les gestionnaires d'événements dans MainView. L'application principale (MedicalTreatmentApp) initialise la base et lance la vue.

Cette séparation permet une maintenance aisée et une évolutivité future.

3.2 Diagramme de classes UML

Le diagramme de classes ci-dessous illustre les principales entités et leurs relations.

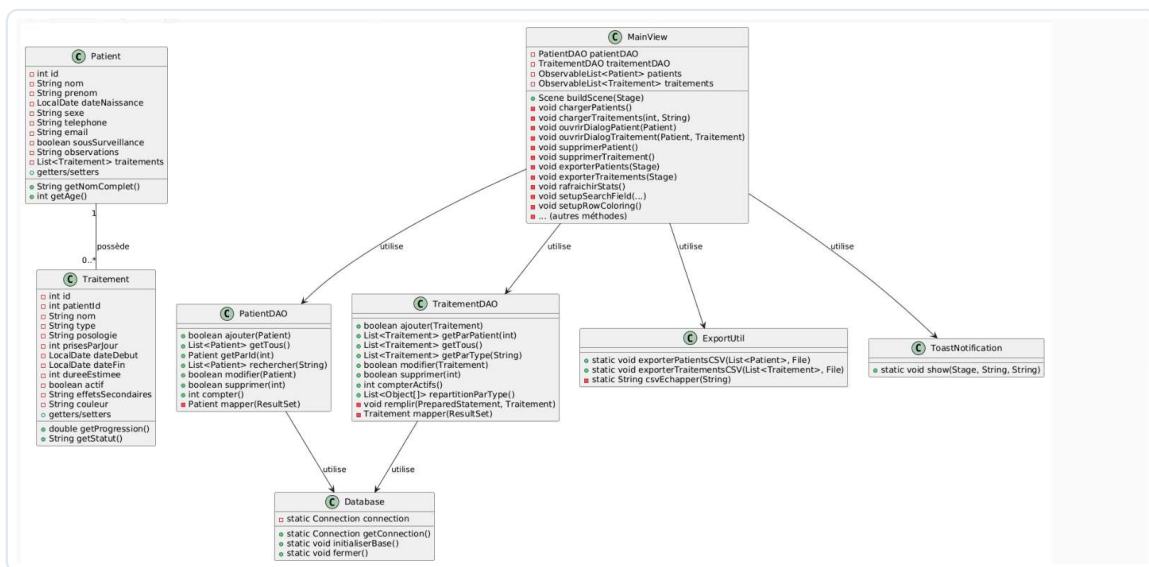


Figure 1 – Diagramme de classes de l'application.

Description :

- La classe Patient possède une liste de Traitement (relation 1-N).
- Les DAO (PatientDAO, TraitementDAO) dépendent de Database pour la connexion.
- MainView orchestre l'interface et fait appel aux DAO, à ExportUtil et à ToastNotification.

3.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les fonctionnalités offertes à l'utilisateur.

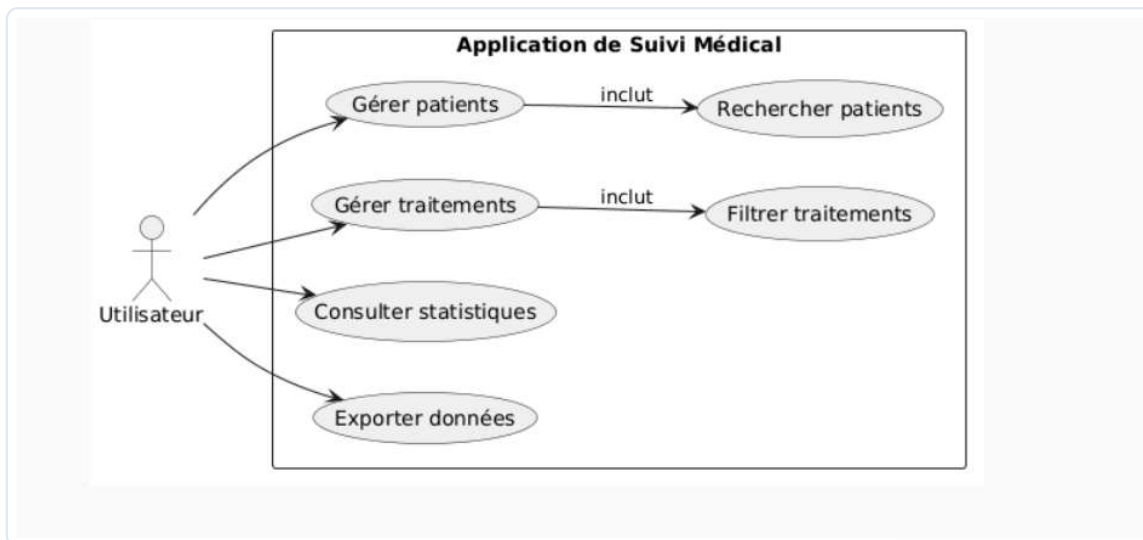


Figure 2 – Diagramme de cas d'utilisation.

Description : L'utilisateur (médecin/secrétaire) peut gérer les patients (incluant la recherche) et les traitements (incluant le filtrage), consulter les statistiques et exporter les données.

3.4 Diagramme de séquence (ajout d'un patient)

Ce diagramme détaille l'interaction entre les composants lors de l'ajout d'un patient.

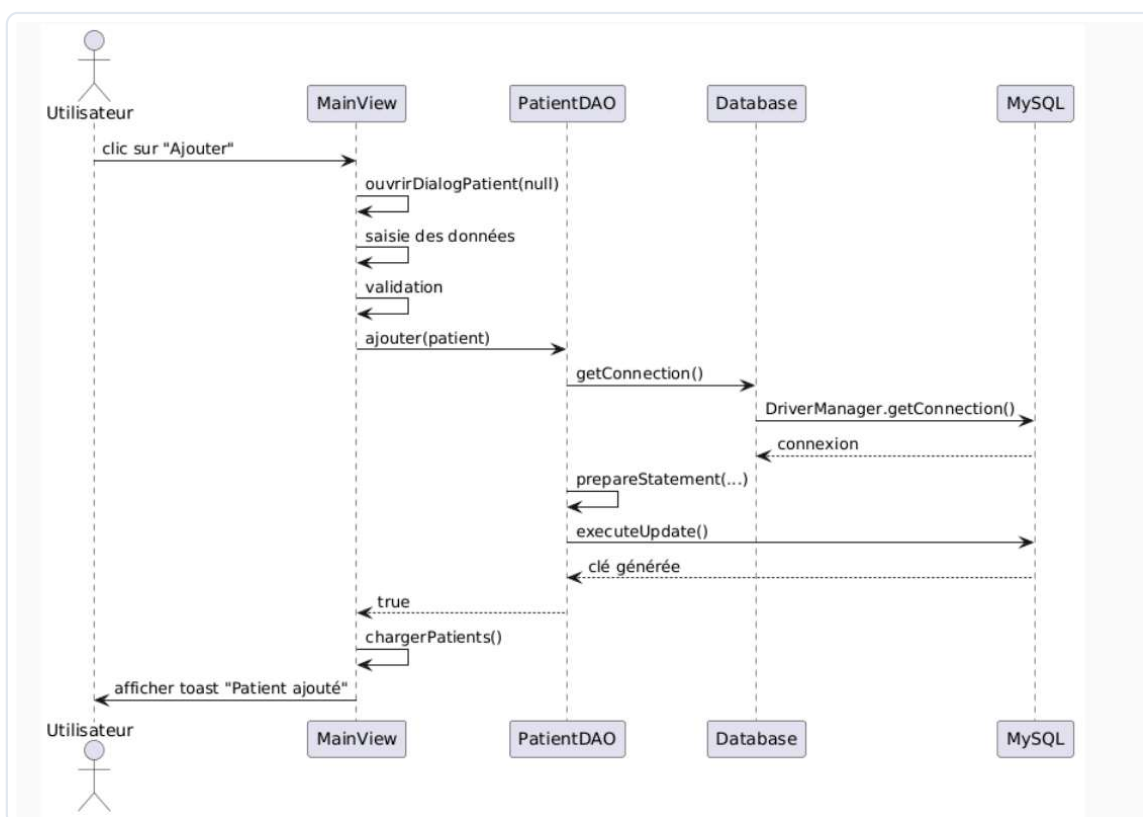


Figure 3 – Diagramme de séquence pour l'ajout d'un patient.

Description : L'utilisateur remplit le formulaire, la vue valide, appelle le DAO qui ouvre une connexion, exécute la requête, récupère l'ID généré et retourne le succès. La vue rafraîchit la liste et affiche une notification.

4. Environnement de travail

4.1 Outils et technologies utilisés

Outil / Technologie	Rôle
Java 17	Langage de programmation
JavaFX 17	Bibliothèque pour l'IHM
MySQL 8	SGBD
MySQL Connector/J	Pilote JDBC
IntelliJ IDEA	IDE
WAMP Server	Serveur local MySQL
Git	Contrôle de version

4.2 Structure du projet

```
src/com/medical/  
├── model/  
│   ├── Patient.java  
│   └── Traitement.java  
├── dao/  
│   ├── Database.java  
│   ├── PatientDAO.java  
│   └── TraitementDAO.java  
├── view/  
│   ├── MainView.java  
│   ├── ToastNotification.java  
│   └── medical-theme.css  
├── util/  
│   └── ExportUtil.java  
└── MedicalTreatmentApp.java (point d'entrée)
```

5. Répartition des tâches

5.1 Tâches de l'étudiant 1 – EL-GHOMRY Youssef

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de la base de données et du schéma	Database.java	Terminé
Implémentation des classes modèles	Patient.java, Traitement.java	Terminé
Implémentation du PatientDAO (CRUD complet)	PatientDAO.java	Terminé
Création de l'onglet Patients (vue, recherche, tableau, détails)	MainView.java (partie patients)	Terminé
Ajout des raccourcis clavier et de la gestion des erreurs	MainView.java	Terminé
Rédaction du rapport (sections Conception, Explication du code)	rapport	Terminé

5.2 Tâches de l'étudiant 2 – DAOUDI Mouad

Tâche réalisée	Fichier(s) concerné(s)	Statut
Implémentation du TraitementDAO (CRUD, filtrage, statistiques)	TraitementDAO.java	Terminé
Création de l'onglet Traitements (split pane, liste patients, tableau, filtres)	MainView.java (partie traitements)	Terminé
Développement de la fonctionnalité d'export CSV	ExportUtil.java	Terminé
Implémentation des notifications Toast	ToastNotification.java	Terminé
Conception du thème CSS et de l'interface statistiques	medical-theme.css, MainView.java (stats)	Terminé
Rédaction du rapport (sections Introduction, Environnement, Interfaces)	rapport	Terminé

6. Explication du code

6.1 Connexion à la base de données – Database.java

La classe Database implémente un **singleton** pour la connexion MySQL. Elle charge le driver JDBC, établit la connexion avec les paramètres définis (URL, utilisateur, mot de passe) et fournit une méthode `initialiserBase()` qui crée les tables `patients` et `traitements` si elles n'existent pas. Les requêtes de création utilisent des contraintes d'intégrité référentielle (`FOREIGN KEY ... ON DELETE CASCADE`). La méthode `fermer()` libère la connexion proprement.

6.2 Les classes modèles

- **Patient** : contient les attributs d'un patient, des méthodes utilitaires (`getNomComple()`, `getAge()`) et une liste de traitements (non persistée directement).
- **Traitement** : modélise un traitement avec une méthode `getProgression()` qui calcule l'avancement en fonction des dates de début et fin. Le statut est dérivé de l'attribut `actif`.

6.3 Les classes DAO

Les DAO (Data Access Object) encapsulent les opérations SQL :

- **PatientDAO** : méthodes d'ajout, lecture (tous, par ID, recherche par mot-clé), modification, suppression et comptage. Le mapping `ResultSet → Patient` est réalisé par une méthode privée `mapper()`.
- **TraitementDAO** : similaire avec des méthodes spécifiques : `getParPatient(int)`, `getParType(String)`, `compterActifs()`, `repartitionParType()` pour les statistiques. Les requêtes préparées protègent contre les injections SQL.

6.4 La vue principale – MainView.java

MainView construit l'intégralité de l'interface en code Java (sans FXML). Elle utilise :

- **TableView** pour afficher les patients et les traitements.
- **ListView** pour la sélection du patient dans l'onglet Traitements.
- **FilteredList** pour la recherche en temps réel.
- **Accordion** pour afficher les détails d'un patient sélectionné.
- **Dialogs** (JavaFX Dialog) pour les formulaires d'ajout/modification, avec validation des champs.
- **Spinner** avec `TextFormatter` pour la saisie des prises par jour.
- **ColorPicker** pour la couleur du traitement.

Les événements sont gérés par des lambdas et des écouteurs de propriétés. Des animations (`fadeIn`) sont appliquées lors du chargement des détails.

6.5 La fonctionnalité d'export

ExportUtil propose deux méthodes statiques pour générer des fichiers CSV. Elles échappent les champs contenant des virgules, des guillemets ou des sauts de ligne pour respecter le format CSV standard.

6.6 Les notifications Toast

ToastNotification affiche une popup temporaire (3 secondes) en bas à droite de la fenêtre, avec un message et une couleur selon le type (success, error, warning, info). Une transition de fondu est utilisée pour disparaître progressivement.

7. Interfaces de l'application

7.1 Fenêtre principale

La fenêtre comporte :

- Un **en-tête** avec le titre, une icône et des chips (Clinique, MVC, JavaFX).
- Une **barre de menus** avec les options Fichier (Export, Quitter), Patients (Ajouter, Modifier, Supprimer), Aide (À propos).
- Un **tab-pane** avec quatre onglets : Patients, Traitements, Stats, Param.
- Une **barre de statut** en bas affichant l'état et le nombre de patients.

7.2 Gestion des patients (onglet Patients)

- Barre d'outils : champ de recherche (filtre dynamique), boutons Ajouter, Modifier, Supprimer, Rafraîchir.
- Tableau listant les patients (ID, Nom, Prénom, Âge, Sexe, Téléphone, Surveillance). Les lignes des patients sous surveillance sont surlignées en rouge pâle.
- Section **Détail patient** (accordéon) : affiche les informations complètes du patient sélectionné, y compris le nombre de traitements associés.

7.3 Gestion des traitements (onglet Traitements)

Le panneau est divisé en deux parties :

- **Gauche** : liste des patients avec recherche, permettant de sélectionner un patient.
- **Droite** : tableau des traitements du patient sélectionné, avec les colonnes : Nom, Type, Posologie, Prises/j, Début, Fin, Statut, Progression (barre de progression). Un filtre par type est disponible.

Les boutons Ajouter, Modifier, Supprimer, Rafraîchir agissent sur les traitements.

7.4 Tableau de bord – Statistiques (onglet Stats)

Trois cartes affichent :

- Nombre total de patients.
- Nombre de traitements actifs.
- Nombre de patients sous surveillance.

Un résumé textuel et un bouton d'actualisation complètent la vue.

7.5 Menus et dialogues

Les dialogues d'ajout/modification utilisent des grilles avec labels et champs. Pour les traitements, un Spinner éditable permet de saisir directement le nombre de prises par jour,

grâce à un TextFormatter qui synchronise la valeur.

7.6 Export de données

Les menus « Exporter patients » et « Exporter traitements » ouvrent un sélecteur de fichier pour enregistrer un CSV. Une notification confirme le succès de l'opération.

8. Conclusion

Le projet « Suivi de Traitements Médicaux » répond pleinement aux exigences fonctionnelles et non fonctionnelles définies en début de semestre. L'utilisation conjointe de JavaFX et MySQL, couplée à une architecture MVC et DAO, a permis de réaliser une application robuste, ergonomique et facilement maintenable.

Les principales difficultés rencontrées concernaient la gestion des composants JavaFX avancés (Spinner édition, liaison des données) et l'intégration du CSS pour un rendu professionnel. Ces obstacles ont été surmontés grâce à une documentation approfondie et à des tests itératifs.

Des améliorations futures pourraient inclure :

- L'ajout d'un module de gestion des utilisateurs (authentification).
- La possibilité d'importer des données depuis un fichier CSV.
- Des graphiques plus détaillés (évolution des traitements dans le temps).
- Une version web ou mobile.

En conclusion, ce projet a permis de renforcer les compétences en développement JavaFX, en conception de bases de données et en méthodologie de projet en binôme.

9. Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Oracle JDBC API : <https://docs.oracle.com/javase/8/docs/api/java/sql/package-summary.html>
- Guide JavaFX CSS : <https://docs.oracle.com/javafx/2/api/javafx/scene/doc-files/cssref.html>
- PlantUML pour les diagrammes : <https://plantuml.com/>

Rapport réalisé dans le cadre du module Développement Java IHM – ENSAO, GI3 2025/2026.